Creative Automation Hub - MVP Design

Overview

AI-powered platform to automate creative content generation at scale.

Core Features (MVP)

1. Text Content Generator

Blog posts, social media captions, ad copy Multiple variants per request Tone/style customization 2. Image Generator

Text-to-image (AI generated) Template-based designs (canva-like) Batch generation 3. Brand Kit

Store colors, fonts, logos Auto-apply to all generated content Style consistency 4. Asset Library

Store generated content Search, filter, organize Version history 5. Export & Share

Multiple formats (PNG, JPG, PDF, MP4) Direct social media posting Download as ZIP Tech Stack

Backend:
- **Go (Golang)** - API, routing, file handling, WebSockets
- **Python Workers** - AI model inference
- PostgreSQL (metadata)
- Redis (caching, job queue)
- S3 (asset storage)

AI Services:
- Groq/Claude (text generation)
- Stable Diffusion (images)
- OpenAI DALL-E (alternative)

Frontend:
- **Next.js 15** + Tailwind CSS
- Fabric.js (canvas editing)
- Real-time updates (WebSockets)

Why Golang:
- 10x faster API responses
- Parallel batch processing
- Better resource efficiency
- Built-in concurrency

MVP User Flow

```
1. User creates project
2. Uploads brand kit (colors, logo)
3. Selects content type (post, ad, blog)
4. Provides brief/keywords
5. AI generates 10 variants
6. User selects/edits favorites
7. Export in desired formats
```

Project Structure

```
creative-automation-hub/
■■■ backend-go/
■   ■■■ cmd/
■   ■   ■■■ server/         # Main entry
■   ■■■ internal/
■   ■   ■■■ handlers/       # HTTP handlers
■   ■   ■■■ services/       # Business logic
■   ■   ■■■ models/         # Data models
■   ■   ■■■ workers/        # Job queue
■   ■■■ pkg/
■   ■   ■■■ client/         # Python AI client
■   ■■■ go.mod
■   ■■■ main.go
■■■ ai-workers/             # Python
■   ■■■ text_generator.py
■   ■■■ image_generator.py
■   ■■■ requirements.txt
■■■ frontend/               # Next.js
■   ■■■ app/                # App router
■   ■■■ components/
■   ■■■ lib/
■   ■■■ package.json
■■■ README.md
```

Database Schema

Projects
- id, user_id, name, created_at

BrandKits
- id, project_id, colors, fonts, logo_url

GeneratedAssets
- id, project_id, type, content, url, metadata

Templates
- id, category, dimensions, preview_url

API Endpoints

```
POST /api/generate/text       # Generate text content
POST /api/generate/image      # Generate images
POST /api/brand-kit           # Save brand kit
GET  /api/assets              # List assets
POST /api/export              # Export content
```

Timeline

Backend API: 2 days AI Integration: 1 day Frontend Dashboard: 3 days Testing: 1 day Total: 1 week Next Steps

Set up project structure Implement text generation (Groq) Add image generation (Stable Diffusion API) Build React dashboard Add brand kit management Deploy MVP Ready to start?